PORTFOLIO · HBM DIGITAL DESIGN (RTL DESIGN)

Mini DRAM Controller

요청 큐 + 뱅크 병렬 스케줄러 — 스펙 · RTL · 검증 · 성능 / PPA 분석

지원 직무	SK hynix · HBM Digital Design (RTL Design)
지원자	용현 권
저장소	`github.com/yhkwon-git/mini-dram-controller` · commit `6049e5a` (2026-10-28)
사용 도구	SystemVerilog · Verilator 5.052 · Yosys 0.52 · sv2v · OpenSTA 3.1 (Nangate45) · Python

파형 · 커버리지 · 합성 수치는 모두 저장소를 수정하지 않고 2026-10-31에 다시 실행해 얻은 결과다. 실험용 TB와 패치는 저장소 밖 스크래치 폴더에만 있다.

문제: 같은 요청도 뱅크 상태에 따라 지연이 2배 넘게 달라진다

DRAM 컨트롤러의 일은 명령 간격 규칙을 클럭 단위로 지키면서, 그 대기를 다른 뱅크의 작업으로 가리는 것

왜 이 프로젝트인가

- 같은 bank 0에서 page hit RD는 6클럭, row conflict RD는 14클럭 (이번 재실행 측정). PRE · ACT 대기를 다른 뱅크 명령으로 채울 수 있느냐가 처리량을 정한다.
- v0는 요청을 하나씩 처리하는 단일 FSM. v1(6049e5a)은 깊이 4 큐 + 뱅크별 타이밍 엔진 ×4 + 클럭당 명령 1개를 고르는 중재기로 바꿨다 — 이 덱은 그 전환의 득실을 잰다.
- 타이밍은 자체 축소 모델(tRCD 3 · tRP 3 · tRAS 6 · tRC 9 · tRFC 12 · tREFI 64 클럭). JEDEC 값이 아니라 구조와 규칙을 보는 스터디다.



범위 — docs/spec.md

host_if → mem_ctrl → dram_model, 단일 동기 클럭 · 비동기 리셋. 실제 PHY · 다중 랭크 · ECC · 사인오프 PPA는 저장소에 없고, 여기서도 만들어 넣지 않는다.

v0 (0cfe062) IDLE에서 요청 1개를 래치 → PRE/ACT/RD/WR을 순서대로
v1 (6049e5a) 큐가 받고, IDLE 뱅크로 dispatch, 엔진들이 겹쳐 진행

5 파일 · 742줄
RTL 5파일 (주석 · 빈 줄 제외 626줄) · mem_ctrl 355줄 · TB 241줄

PASS · 위반 0
저장소 TB 4개 시나리오 · 요청 18개 · VCD로 타이밍 규칙 7개 검사

2.06× 버스트
4뱅크 쓰기 4개: 첫 수락 → 마지막 응답 17클럭 (v0 35클럭)

373 MHz
면적 9,819 µm² · Nangate45 typical · 배선 없음 · mem_ctrl만 · 참고치

스펙: 주소 맵 · 타이밍 8개 · 명령 6개를 dram_pkg 한 곳에 고정

docs/spec.md와 rtl/dram_pkg.sv가 같은 값 · 오른쪽 표의 '지키는 곳'은 그 규칙을 실제로 집행하는 RTL 위치

주소 14b → bank · row · col

bank [13:12]	row [11:4]	col [3:0]
4 뱅크	256 row (8b)	16 col (4b)

4 × 256 × 16 = 16,384 워드 × 32b (64 KiB) · TB는 `make_addr(b, r, c) = {b, r, c}`

명령 dram_cmd_e (3b) — 모델 동작

명령	코드	dram_model 동작
NOP	000	없음
ACT	001	bank_open ← 1, bank_row ← row · 열린 뱅크의 다른 row면 access_err
RD	010	tCL 2 뒤 data_valid · rdata · 닫힘/다른 row면 access_err, 데이터 0
WR	011	row가 맞으면 메모리에 쓰기 · 아니면 access_err, 쓰지 않음
PRE	100	bank_open ← 0 (뱅크 하나)
REF	101	모든 뱅크를 닫고 tRFC 12 동안 busy

호스트: `req` valid/ready · `req_write` 1 = WR · 응답은 요청 순서로 하나씩, WR도 응답을 낸다 (데이터 0) · `rsp_err`는 v1에서 0 고정

타이밍 (클럭) — 누가 지키는가

파라미터	값	의미	지키는 곳 (RTL)
tRCD	3	ACT → RD/WR	`b_timer` · ST_ACT_WAIT
tRP	3	PRE → ACT	`b_timer` · ST_PRE_WAIT
tRAS	6	ACT → PRE 최소	`ras_rem` → want_pre
tRC	9	같은 뱅크 ACT → ACT	`rc_rem` → want_act
tWR	2	WR 회복	`b_timer` · ST_WR_WAIT
tCL	2	RD → data_valid	모델 `rd_cnt`
tRFC	12	REF busy	모델 `rfc_cnt` → busy
tREFI	64	리프레시 주기	`refi_cnt` → ref_due

문서 ↔ RTL 차이 spec §4의 '수락은 ST_IDLE에서만'과 §5 FSM 그림, scheduler-throughput.md 머리말은 단일 FSM을 'v1 single-outstanding'이라 부르는데, 이 덱의 v0에 해당한다. v1은 큐가 수락하고 뱅크마다 상태를 가진다. `ST_REF` · `ST_REF_WAIT` · `ST_ERROR`는 enum에만 있고 쓰이지 않는다 (REF는 `ref_active` 플래그). 이 덱의 숫자는 모두 v1 RTL을 다시 돌려 잰 값이다.

아키텍처: 단일 FSM(v0) → 큐 + 뱅크 병렬 엔진(v1)

두 버전을 같은 TB · 같은 측정 스크립트로 돌려 비교 — v0는 0cfe062/64e73e0의 mem_ctrl, v1은 6049e5a

	v0 단일 FSM	v1 큐 + 뱅크 엔진 ×4
수락	ST_IDLE에서 요청 1개	큐 빈 슬롯이 있으면 (깊이 4) · REF 대기 중엔 차단
상태	state 1개 · 타이머 1개	뱅크마다 `b_state` · `b_timer` + 열린 row 섀도
명령 발행	대기 상태를 나오는 클럭에 바로	뱅크 `want_*` → 중재기 PRE > ACT > WR > RD, 클럭당 1개
응답 순서	하나씩이라 자연히 in-order	tag 16개 응답 버퍼 → `tag_retire` 순서로 반환
리프레시	IDLE에서 REF 우선	`ref_due` → 수락 차단 → 전 뱅크 IDLE · 큐 0이면 REF
4뱅크 버스트	35클럭	**17클럭 (2.06×)**
hit RD 지연	4클럭	6클럭 — 구조 비용 +2 (17쪽)
면적 · FF	1,208 μm² · 131	9,819 μm² · 1,072 (8.1×, 20쪽)

High Speed Link Lite의 MC는 큐 head만 발행하는 in-order 구조였다. 이 프로젝트는 그 다음 단계 — 뱅크 병렬 · tag 기반 응답 재정렬 · 리프레시 스케줄링 — 를 다룬다.

Open-page 정책

응답 뒤에도 row를 열어 둔다. 같은 row면 바로 RD/WR, 다른 row면 PRE → ACT. 닫히는 때는 conflict와 REF뿐

Oldest-tag dispatch

큐에서 '대상 뱅크가 IDLE인 항목' 중 가장 오래된 tag를 보낸다. `q_tag - tag_retire`로 랩어라운드에도 나이 비교

Drain-then-REF

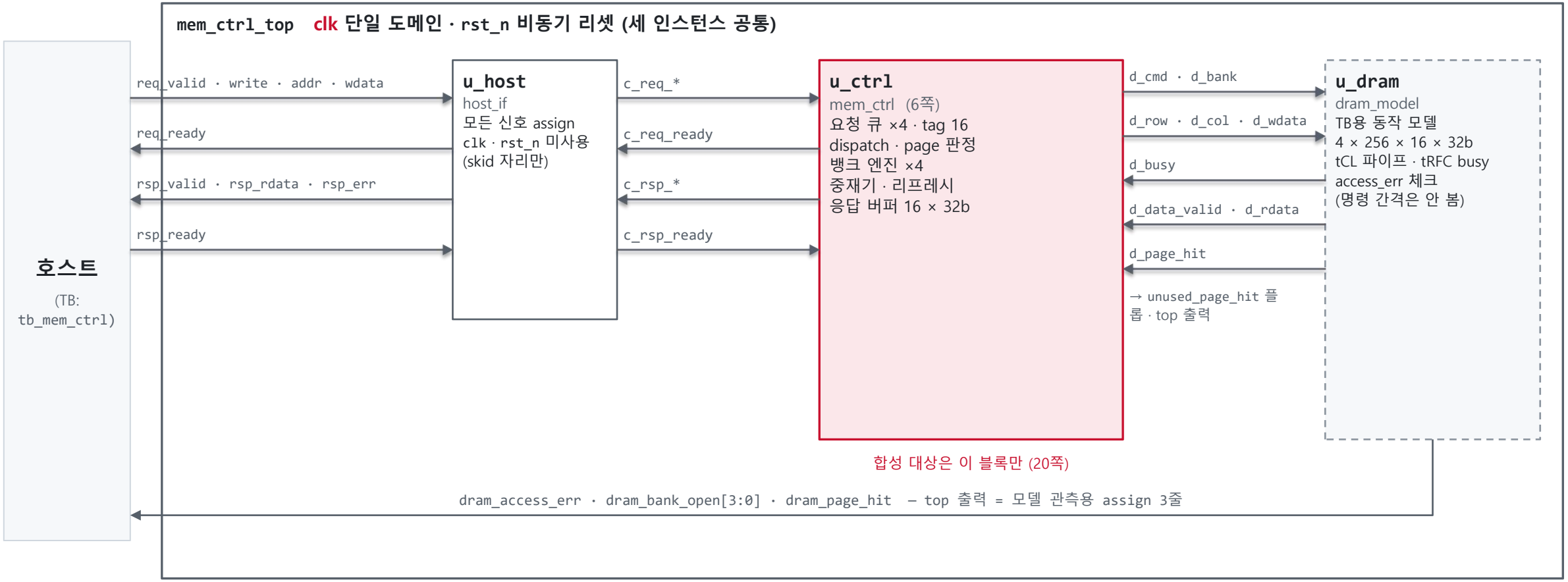
`ref_due`면 새 수락을 막고, 진행 중 뱅크와 큐가 다 비면 REF. 단순한 대신 REF 직전 열린 row를 모두 잃는다

섀도 open-row

컨트롤러가 `bank_open` · `bank_row`와 tRAS/tRC 잔여를 직접 들고 판단. 모델의 `page_hit` 출력은 쓰지 않는다

Top level: host_if → mem_ctrl → dram_model, 단일 클럭 동기 설계

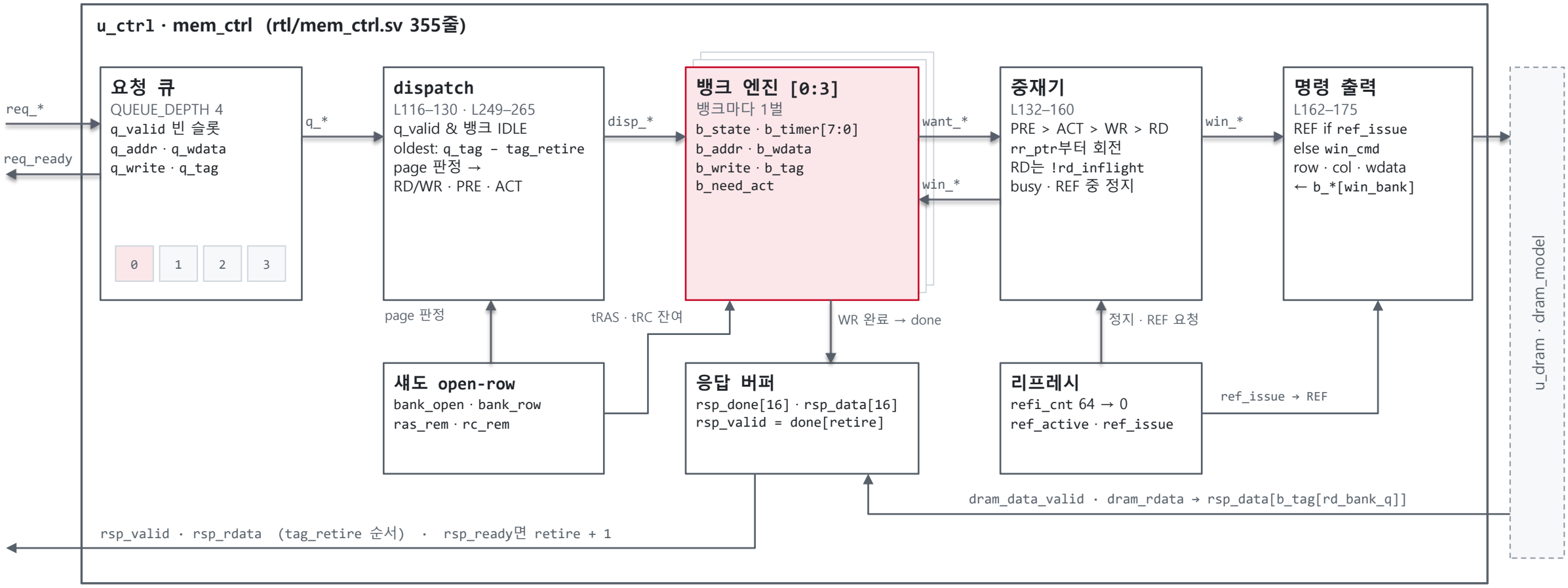
rtl/mem_ctrl_top.sv의 인스턴스 · 포트 · 넷 이름 그대로



범례 req_addr[13:0] = {bank, row, col} · req_wdata · rsp_rdata 32b · d_cmd = dram_cmd_e 3b · c_*는 호스트 쪽 신호와 1:1 (host_if L39–48)
hit 판정은 컨트롤러의 섀도 bank_open / bank_row가 한다 — 모델의 page_hit은 관측용. 실선 = RTL, 점선 = TB 모델

mem_ctrl: 큐 → dispatch → 뱅크 엔진 ×4 → 중재기 → 명령 1개/클럭

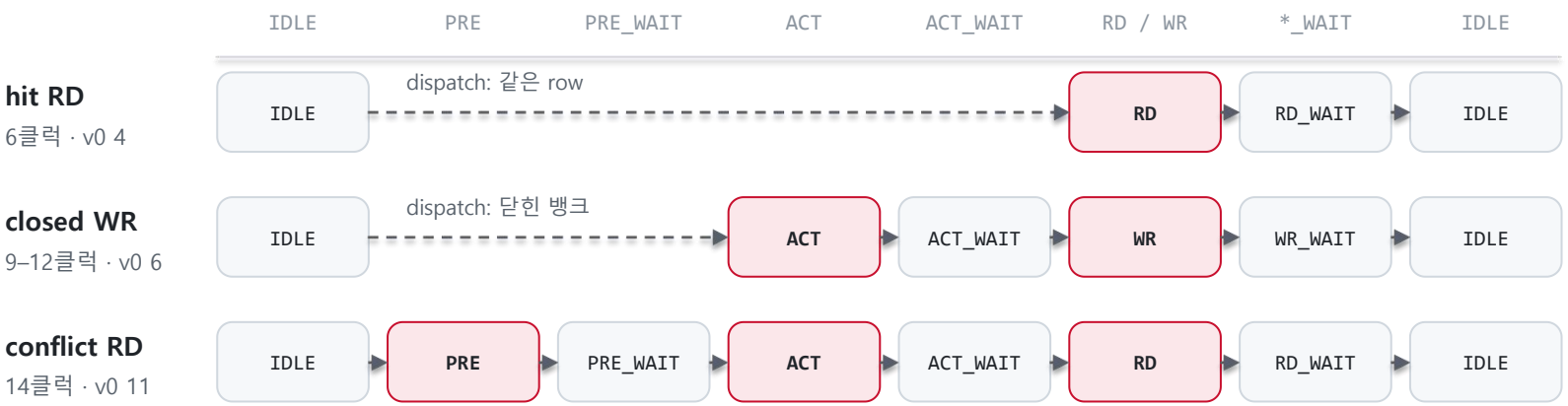
always_comb 하나가 want_* · ref_issue · req_ready · disp_* · win_*을 만들고, always_ff 하나가 모든 상태를 갱신



`req_ready = free_found && !ref_due && !ref_active` (L113) — 응답 버퍼 여유는 보지 않는다 → 18쪽 버그. 임계 경로는 `tag_retire` → dispatch 나이 비교 → `b_state` (20쪽)

뱅크 엔진: 상태를 지나는 세 가지 경로와 실제 지연

열 = b_state, 행 = dispatch 때의 page 판정 · 오른쪽 숫자는 이번 재실행에서 잰 수락 → 응답 클럭 (v0 같은 TB)



굵은 상자 = 중재기에 명령을 요청하는 상태 (want_*) · 점선 = 건너뛴 상태

전이	조건	동작
PRE → PRE_WAIT	want_pre = ras_rem == 0 · 중재 승리	PRE 발행 · b_timer ← tRP-1 · bank_open ← 0
PRE_WAIT → ACT	b_timer == 0 & b_need_act	(need_act = 0이면 IDLE — TB에서 미실행)
ACT → ACT_WAIT	want_act = rc_rem == 0 · 중재 승리	ACT 발행 · b_timer ← tRCD-1 · tRAS/tRC 장전
ACT_WAIT → RD/WR	b_timer == 0	상태만 바뀜 → 다음 클럭 중재 (17쪽 +1)
RD → RD_WAIT → IDLE	중재 승리 & !rd_inflight → dram_data_valid	rsp_data[b_tag] ← rdata · rd_inflight ← 0
WR → WR_WAIT → IDLE	중재 승리 → b_timer == 0	b_timer ← tWR-1 → rsp_done[b_tag] ← 1

u_dram · dram_model
TB용 동작 모델 (rtl/dram_model.sv)
모델 쪽 bank_open · bank_row ×4
RD: tCL 2 파이프 → data_valid · rdata
REF: 전 뱅크 닫음, tRFC 12 동안 busy
access_err: 열린 뱅크에 다른 row ACT (L110), 닫힌/다른 row RD · WR (L122 · L132)
명령 간격(tRCD 등)은 검사하지 않음

- 엔진 4개가 timer · want를 각자 계산하고 명령 버스 하나만 공유 → 15쪽에서 8클럭 연속 발행
- ST_REF · ST_REF_WAIT · ST_ERROR는 enum에만 있음
- hit WR 경로 (dispatch → ST_WR, L259)는 TB에서 한 번도 지나가지 않음 (13쪽)
- 모델이 타이밍을 안 보므로 VCD 체커를 따로 만들어 위반 0을 확인 (12 · 13쪽)

수락과 dispatch: 빈 슬롯이면 받고, IDLE 뱅크 중 가장 오래된 요청부터

req_ready는 큐와 리프레시만 본다 · 뱅크가 바쁜 요청은 큐에서 기다리고, 다른 뱅크 요청이 먼저 나갈 수 있다

rtl/mem_ctrl.sv L113–114, L116–130

```
113  req_ready = free_found && !ref_due && !ref_active;                          ①
114  q_push    = req_valid && req_ready;

116  // Prefer oldest tag among ready-to-dispatch entries
117  disp_valid = 1'b0;
118  disp_bank  = '0;
119  disp_slot  = '0;
120  if (!ref_active && !ref_issue) begin                                        ②
121    for (bi = 0; bi < QUEUE_DEPTH; bi++) begin
122      if (q_valid[bi] && (b_state[addr_bank(q_addr[bi])] == ST_IDLE)) begin   ③
123        if (!disp_valid || (q_tag[bi] - tag_retire) < (q_tag[disp_slot] - tag_retire)) begin  ④
124          disp_valid = 1'b1;
125          disp_bank  = addr_bank(q_addr[bi]);
126          disp_slot  = bi[QUEUE_AW-1:0];
127        end
128      end
129    end
130  end
```

rtl/mem_ctrl.sv L249–265

```
249  // Dispatch
250  if (disp_valid) begin
251    q_valid[disp_slot] <= 1'b0;                                               ⑤
252    b_addr[disp_bank]  <= q_addr[disp_slot];
253    b_wdata[disp_bank] <= q_wdata[disp_slot];
254    b_write[disp_bank] <= q_write[disp_slot];
255    b_tag[disp_bank]   <= q_tag[disp_slot];
256    b_need_act[disp_bank] <= 1'b0;
257    if (bank_open[disp_bank] &&                                               ⑥
258        (bank_row[disp_bank] == addr_row(q_addr[disp_slot])))
259      b_state[disp_bank] <= q_write[disp_slot] ? ST_WR : ST_RD;
260    else if (bank_open[disp_bank]) begin                                      ⑦
261      b_need_act[disp_bank] <= 1'b1;
262      b_state[disp_bank]    <= ST_PRE;
263    end else
264      b_state[disp_bank] <= ST_ACT;                                           ⑧
265  end
```

① req_ready = 빈 슬롯 & !ref_due & !ref_active — 응답 버퍼 여유는 보지 않는다 (18쪽 버그의 출발점)

② REF를 내는 중이거나 진행 중이면 dispatch 정지 — REF 앞에서 큐가 비도록

③ 대상 뱅크가 ST_IDLE인 항목만 후보 → 바쁜 뱅크 요청은 남고 다른 뱅크가 추월 (TB는 큐 점유 최대 1이라 미관찰)

④ q_tag − tag_retire (4b 모듈러)로 나이 비교 → tag가 0으로 돌아가도 oldest 유지

⑤ dispatch 클럭에 슬롯을 비우고 주소 · 데이터 · tag를 뱅크 엔진 b_* 로 복사

⑥ 섀도 bank_open / bank_row로 hit 판정 → 곧바로 ST_RD / ST_WR (ACT 생략)

⑦ 열려 있지만 다른 row → ST_PRE + b_need_act → PRE_WAIT 뒤 자동으로 ACT

⑧ 닫힌 뱅크 → ST_ACT. 세 경로가 7쪽의 hit / conflict / closed

중재와 발행: 뱅크는 '원한다'만 말하고, 중재기가 클럭당 하나를 고른다

want_* (조합) → PRE > ACT > WR > RD + rr_ptr 회전 → 발행 클럭에 타이머 preload와 섀도 갱신

rtl/mem_ctrl.sv L92–97, L135–141, L156–157

```
92  case (b_state[bi])
93    ST_PRE: want_pre[bi] = (ras_rem[bi] == 0);                                  ①
94    ST_ACT: want_act[bi] = (rc_rem[bi] == 0);
95    ST_RD:  want_rd[bi]  = 1'b1;
96    ST_WR:  want_wr[bi]  = 1'b1;
97    default: ;
    ------------------------------------------------------------------------
135 if (!dram_busy && !ref_active && !ref_issue) begin                           ②
136   for (bi = 0; bi < NUM_BANKS; bi++) begin
137     automatic int b = (int'(rr_ptr) + bi) % NUM_BANKS;                        ③
138     if (!win_valid && want_pre[b]) begin                                      ④
139       win_valid = 1'b1; win_bank = b[$clog2(NUM_BANKS)-1:0]; win_cmd = CMD_PRE;
140     end
141   end
    ------------------------------------------------------------------------
156 if (!win_valid && want_rd[b] && !rd_inflight) begin                            ⑤
157   win_valid = 1'b1; win_bank = b[$clog2(NUM_BANKS)-1:0]; win_cmd = CMD_RD;
```

rtl/mem_ctrl.sv L285–296, L323–324

```
285 CMD_PRE: begin
286   bank_open[win_bank] <= 1'b0;
287   b_timer[win_bank]   <= 8'(tRP > 0 ? tRP - 1 : 0);
288   b_state[win_bank]   <= ST_PRE_WAIT;
289 end
290 CMD_ACT: begin
291   bank_open[win_bank] <= 1'b1;                                              ⑥
292   bank_row[win_bank]  <= addr_row(b_addr[win_bank]);
293   ras_rem[win_bank]   <= tRAS[$bits(ras_rem[0])-1:0];
294   rc_rem[win_bank]    <= tRC[$bits(rc_rem[0])-1:0];
295   b_timer[win_bank]   <= 8'(tRCD > 0 ? tRCD - 1 : 0);                       ⑦
296   b_state[win_bank]   <= ST_ACT_WAIT;
    ------------------------------------------------------------------------
323 if (b_state[i] == ST_ACT_WAIT && b_timer[i] == 0)                            ⑧
324   b_state[i] <= b_write[i] ? ST_WR : ST_RD;
```

① want_pre는 tRAS 잔여 0, want_act는 tRC 잔여 0일 때만 — 뱅크마다 자기 카운터

② DRAM busy이거나 REF 중이면 아무것도 발행하지 않음

③ rr_ptr부터 4뱅크를 검사 → 같은 종류끼리 회전 우선 (발행 후 rr_ptr ← win_bank + 1)

④ PRE 루프가 먼저, 이어 ACT · WR · RD 루프 — 행 명령을 먼저 내 다른 뱅크의 tRP/tRCD를 겹친다

⑤ RD는 !rd_inflight일 때만 → 읽기 파이프 1개, RD는 최대 4클럭에 1번 (17쪽)

⑥ ACT 클럭에 섀도 bank_open · bank_row와 tRAS · tRC 잔여를 함께 장전

⑦ b_timer ← tRCD − 1: 대기 종료 클럭에 바로 발행하던 v0 기준. v1에서는 간격이 tRCD + 1

⑧ ACT_WAIT 종료는 상태만 ST_RD/WR로 → 실제 RD/WR는 다음 클럭 중재기가 발행

리프레시는 'busy 뒤 REF', 응답은 tag 순서로 retire

refi_cnt가 0이면 수락을 막고 진행 중 작업을 기다린다 · 완료는 순서와 무관하게 버퍼에, 반환은 tag_retire 순서

rtl/mem_ctrl.sv L101–102, L218–224, L227–230

```
101  ref_due    = (refi_cnt == 0);                                                    ①
102  ref_issue = ref_due && !ref_active && all_banks_idle && (q_count == 0) && !dram_busy;  ②

218  if (ref_active) begin
219    if (!dram_busy) begin                                                         ③
220      ref_active <= 1'b0;
221      refi_cnt   <= tREFI[$bits(refi_cnt)-1:0];
222    end
223  end else if (refi_cnt != 0) begin
224    refi_cnt <= refi_cnt - 1'b1;

227  if (ref_issue) begin
228    ref_active <= 1'b1;
229    for (int i = 0; i < NUM_BANKS; i++) begin
230      bank_open[i] <= 1'b0;                                                       ④
```

rtl/mem_ctrl.sv L178–179, L327–329, L335–337, L342–345

```
178  rsp_valid = rsp_done[tag_retire];                                             ⑤
179  rsp_rdata = rsp_data[tag_retire];

327  if (b_state[i] == ST_WR_WAIT && b_timer[i] == 0) begin
328    rsp_done[b_tag[i]] <= 1'b1;
329    rsp_data[b_tag[i]] <= '0;                                                   ⑥

335  if (rd_inflight && dram_data_valid) begin
336    rsp_done[b_tag[rd_bank_q]] <= 1'b1;
337    rsp_data[b_tag[rd_bank_q]] <= dram_rdata;                                   ⑦

342  if (rsp_valid && rsp_ready) begin                                             ⑧
343    rsp_done[tag_retire] <= 1'b0;
344    tag_retire <= tag_retire + 1'b1;
345  end
```

① `ref_due = refi_cnt == 0` → 같은 클럭부터 req_ready 0 (8쪽 ①)

② REF 조건: 전 뱅크 IDLE & 큐 0 & DRAM idle — 첫 REF는 진행 중 WR 때문에 6클럭 대기 (16쪽)

③ busy가 풀리는 클럭에 `ref_active` 해제, refi_cnt를 64로 다시 장전

④ REF가 모델의 뱅크를 다 닫으므로 섀도도 닫음 → REF 직후 같은 row는 hit이 아니라 closed

⑤ `rsp_valid = rsp_done[tag_retire]` — 가장 오래된 요청이 끝나야 응답. 뒤 요청이 먼저 끝나면 대기

⑥ WR도 응답을 낸다 (데이터 0) → 호스트는 요청 수 = 응답 수로 셀 수 있다

⑦ RD 데이터를 `b_tag[rd_bank_q]` 칸에 쓴다 — 칸이 비었는지 보지 않음 → tag가 한 바퀴 돌면 덮어씀 (18쪽)

⑧ 핸드셰이크 때 done 비트를 지우고 `tag_retire + 1`

DRAM 모델은 규칙 위반을 플래그로 알리고, 상수는 패키지 한 곳에 둔다

dram_model은 상태 · 데이터 · tCL/tRFC만 모델링하고 명령 간격은 검사하지 않는다 — 그 빈칸을 VCD 체커로 메웠다

rtl/dram_model.sv L108–115, L120–125, L130–135

```
108  CMD_ACT: begin
109    // Opening a new row on an already-open bank without PRE is a conflict
110    if (bank_open[bank] && bank_row[bank] != row)                    ①
111      access_err <= 1'b1;
112    bank_open[bank] <= 1'b1;
113    bank_row[bank]  <= row;
114    page_hit        <= 1'b0;
115  end
     ----------------------------------------------------------------
120  CMD_RD: begin
121    page_hit   <= bank_open[bank] && (bank_row[bank] == row);
122    access_err <= ~(bank_open[bank] && (bank_row[bank] == row));     ②
123    rd_active  <= 1'b1;
124    rd_was_hit <= bank_open[bank] && (bank_row[bank] == row);
125    rd_cnt     <= tCL[$bits(rd_cnt)-1:0];                            ③
     ----------------------------------------------------------------
130  CMD_WR: begin
131    page_hit   <= bank_open[bank] && (bank_row[bank] == row);
132    access_err <= ~(bank_open[bank] && (bank_row[bank] == row));
133    if (bank_open[bank] && (bank_row[bank] == row))                   ④
134      mem[mem_index(bank, row, col)] <= wdata;
135  end
```

커버리지 dram_model 분기 18개 중 3개 미실행 — L110 if (ACT 충돌), L133 else (잘못된 WR), L93 else (miss RD). 모델의 에러 경로가 한 번도 발화하지 않았으므로 `access_err count = 0`은 '위반이 없었다'까지만 말한다 (19쪽 E).

rtl/dram_pkg.sv L20–27

```
20  localparam int tRCD = 3;   // ACT -> RD/WR                          ⑤
21  localparam int tRP  = 3;   // PRE -> ACT
22  localparam int tRAS = 6;   // ACT -> PRE (min)
23  localparam int tRC  = 9;   // ACT -> ACT same bank (≈ tRAS+tRP)     ⑥
24  localparam int tRFC = 12;  // REF busy window
25  localparam int tCL  = 2;   // RD -> data_valid
26  localparam int tWR  = 2;   // WR recovery before PRE
27  localparam int tREFI = 64; // refresh opportunity period            ⑦
```

① ACT를 열린 뱅크의 다른 row로 (PRE 누락) → `access_err`

② RD가 닫힌 뱅크나 다른 row로 오면 `access_err` — RD/WR마다 다시 계산

③ tCL 카운트 뒤 `data_valid` 1클럭 → 컨트롤러 RD_WAIT의 종료 조건

④ 잘못된 WR는 메모리에 쓰지 않음 → 데이터 비교로도 드러난다

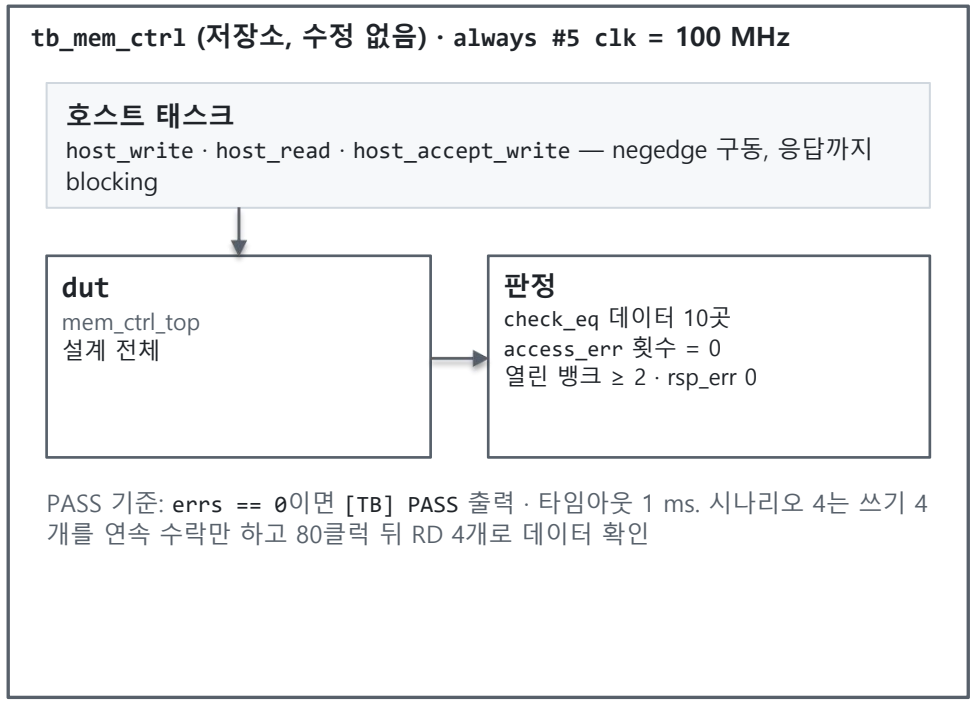
⑤ tRCD · tRP · tRAS → 뱅크 엔진 `b_timer` · `ras_rem`이 사용

⑥ tRC 9 = tRAS 6 + tRP 3 (문서 그대로) · 측정 최소 tRC 23, 위반 0

⑦ tREFI 64 · tRFC 12 → 문서 추정 REF 비중 ≈19%, 이번 측정 18.3%

검증: 저장소 TB 4개 시나리오 + 포트폴리오용 측정 · 실험 계층

저장소 tb_mem_ctrl은 수정 없이 그대로 돌리고, 그 위에 VCD 분석과 별도 실험 TB를 얹었다 — 실험 코드는 저장소 밖



포트폴리오 계층 (스크래치, 저장소 밖)

`tb_pf_top`: tb_mem_ctrl 인스턴스 + `$dumpvars` → full.vcd
`tb_pf_backpressure`: rsp_ready 150클럭 0 + 읽기 20개 (18쪽)
`analyze.py`: VCD → 명령 간격 · 지연 분류 · 엔진 동시성

시나리오	자극	확인
1 page hit	WR b0 r11 → 같은 주소 RD	A5A5_5A5A 일치
2 page miss	b0 r22 WR · RD → 다시 r11 RD	새 row · 원래 row 데이터 모두
3 multi-bank	b1 r33 WR · RD, b0 r44 conflict, b1 재확인	b0 row 변경 뒤 b1 데이터 유지
4 4뱅크 버스트	b0–b3 WR 4개 연속 수락 → 80클럭 뒤 RD 4개	4개 데이터 · 열린 뱅크 ≥ 2
공통	전 구간	access_err 0 · rsp_err 0

이번에 추가한 검사 (저장소 TB가 하지 않는 것)

검사	방법	결과
타이밍 규칙 7개	VCD 명령을 뱅크별로 짝지어 tRCD · tRP · tRAS · tRC · tWR · tCL · tRFC 최소 간격	13쪽
지연 분류	요청마다 hit / closed / conflict와 수락 → 응답 클럭	14–17쪽
v0 A/B	v0 mem_ctrl (64e73e0) + HEAD 모델 · TB로 같은 측정	4 · 17쪽
백프레셔	coverage.md의 gap — rsp_ready를 막고 읽기 20개	18쪽
커버리지 · 합성	Verilator line · branch · toggle, Yosys + OpenSTA	13 · 20쪽

결과: PASS · 타이밍 위반 0 · mem_ctrl 라인 94.3% / 분기 91.9%

Verilator 5.052로 저장소 그대로 재실행 — 빌드 1.04 s · 시뮬 3.155 µs (312클럭) · 명령 ACT 14 · PRE 4 · RD 10 · WR 8 · REF 4

sim/regress.log (make sim과 같은 플래그)

```
=== SCENARIO 1: page hit WR→RD ===
hit RD OK (0xa5a55a5a)
=== SCENARIO 2: page miss (bank0 row change) ===
miss-new RD OK (0xbeef0002)
miss-old RD OK (0xa5a55a5a)
=== SCENARIO 3: multi-bank + bank0 conflict ===
bank1 RD OK (0xc0de0001)
bank0 conflict RD OK (0xd00d0004)
bank1 survived OK (0xc0de0001)
=== SCENARIO 4: queue depth — burst accept 4 banks ===
peak banks open during burst=4
q bank0 OK (0x10000000)
q bank1 OK (0x20000000)
q bank2 OK (0x30000000)
q bank3 OK (0x40000000)
access_err count=0 OK
PASS — hit/miss/multi-bank/queue-parallel OK
```

- `verilator -Wall --timing --sv --binary` (Makefile `sim`과 동일) · lint: 에러 0, UNUSEDSIGNAL 경고 9개 (host_if `clk/rst_n`, mem_ctrl의 루프 변수 `b` 상위 비트 등)
- 파형은 `--trace` 래퍼, 커버리지는 `--coverage-line --coverage-toggle` 별도 빌드 · cocotb 경로는 실행하지 않음

커버리지 (Verilator)

모듈	라인	분기	토글
mem_ctrl	94.3%	91.9%	51.2%
dram_model	100.0%	83.3%	75.7%
host_if	–	–	81.5%
mem_ctrl_top	–	–	82.1%
합계	95.8%	90.2%	63.7%

타이밍 규칙 (VCD 체커)

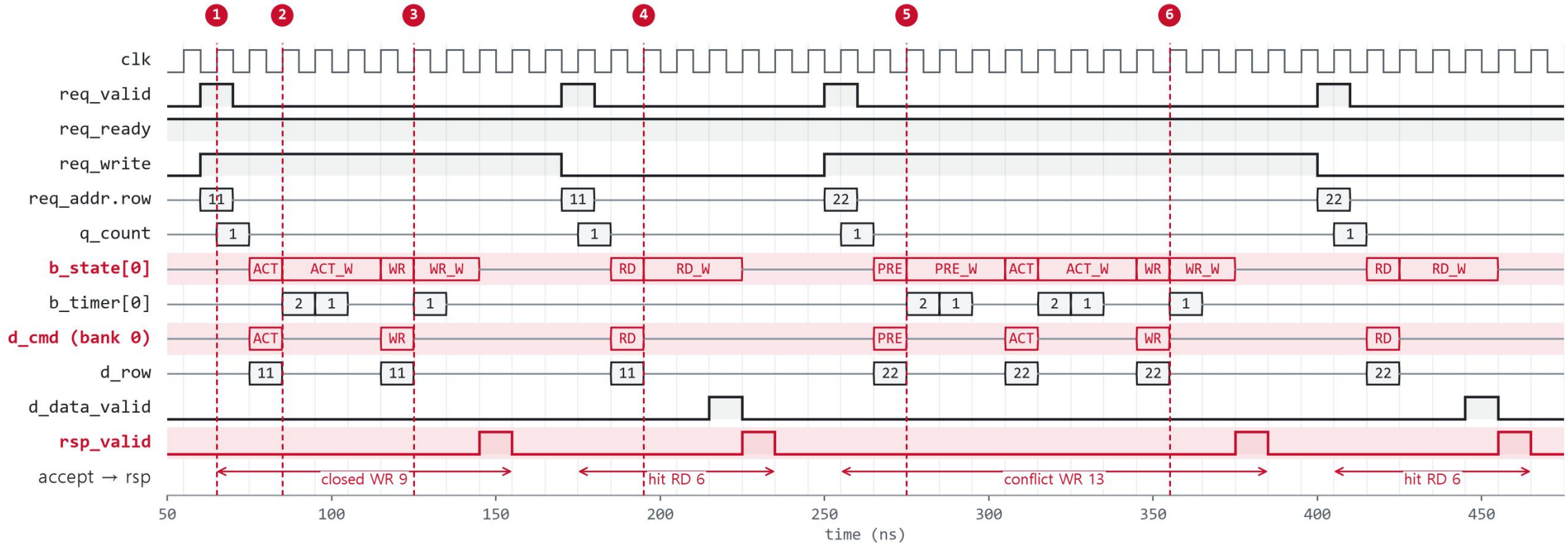
규칙	스펙	측정 최소	횟수	위반
tRCD	3	4	18	0
tRP	3	4	4	0
tRAS	6	19	4	0
tRC	9	23	4	0
tWR	2	15	3	0
tCL	2	2	10	0
tRFC	12	16	3	0

tWR = WR → 같은 뱅크 PRE, tRFC = REF → 다음 명령. 음영: 스펙보다 1클럭 김 (17쪽)

커버리지 구멍 → 다음 테스트

- 큐 점유 최대 1 — 슬롯 1–3 미사용. 태스크가 응답까지 기다리는 blocking 구조라 깊이 4를 시험하지 못함 (coverage.md의 'Depth-4 accept'와 다름)
- write hit (L259) · push와 dispatch 동시 (L271) · PRE_WAIT → IDLE (L315) · 두 후보 나이 비교 (L123 else) 미실행
- 모델 에러 분기 3개 미실행 → access_err 경로에 음성 테스트 없음
- 토글 51.2%: `rsp_data` 16 × 32b 등 넓은 레지스터 대부분이 고정
- L287 · 295 · 299 cond_else, L307 default는 상수 · 도달 불가 분기
- coverage.md의 gap 'Backpressure'를 18쪽 실험으로 닫음 → 버그 발견

page 정책: 같은 bank 0에서 closed 9 · hit 6 · conflict 13클럭



1 65 ns WR b0 row 11 수락 · q_count 1. 뱅크가 닫혀 있어 closed 경로

2 85 ns ACT — 수락 후 2클럭 (큐 등록 → dispatch → 발행)

3 125 ns WR — ACT 후 4클럭 = tRCD 3 + 1 (대기 종료 다음 클럭에 발행)

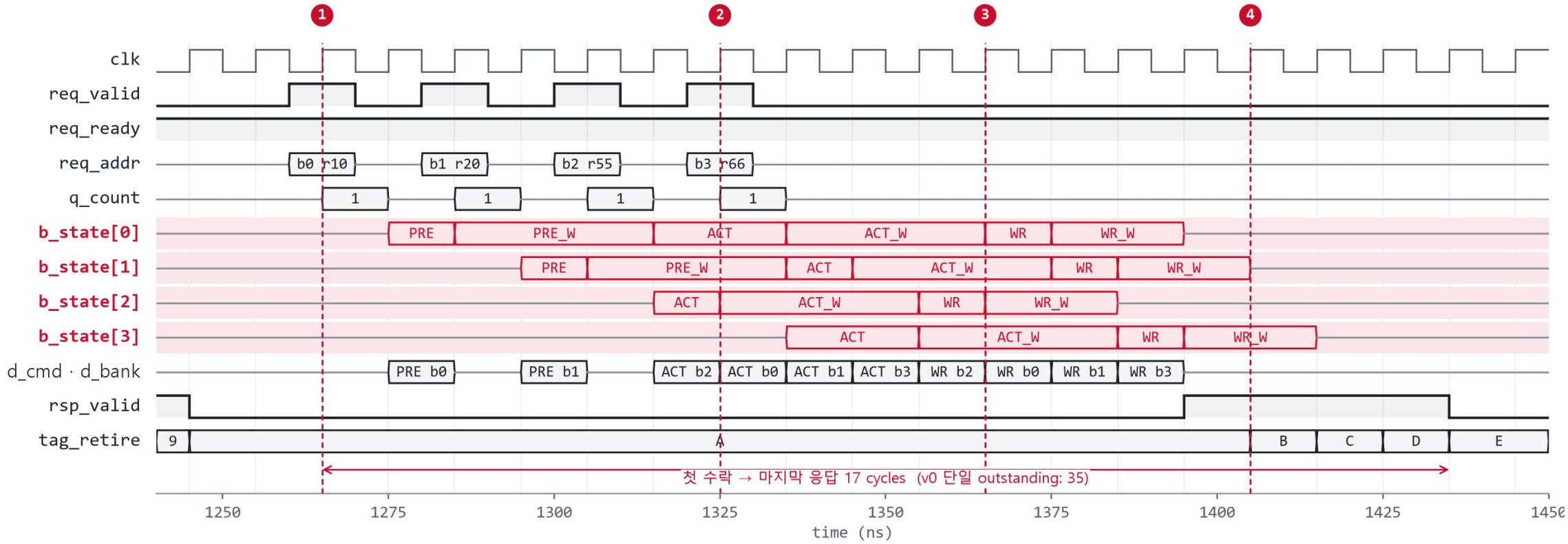
4 195 ns 같은 row RD → ACT 없이 바로 RD, hit 6클럭

5 275 ns row 22 WR → 열린 row 11과 충돌, PRE부터 (`b_need_act`)

6 355 ns PRE → ACT 4 · ACT → WR 4클럭 → conflict 13클럭

tb_mem_ctrl 시나리오 1 · 2, 50–475 ns · 저장소 TB를 `tb_pf_top` 래퍼로 감싸 Verilator `--trace` VCD에서 그린 실제 파형 · 마커는 샘플링 클럭 엣지

4뱅크 버스트: 엔진 4개가 겹쳐 돌고, 응답은 tag 순서로 연속 4클럭



1 1265 ns WR b0 r10 수락. 2클럭 간격으로 b1 · b2 · b3 — 바로 dispatch되어 q_count는 1을 넘지 않음

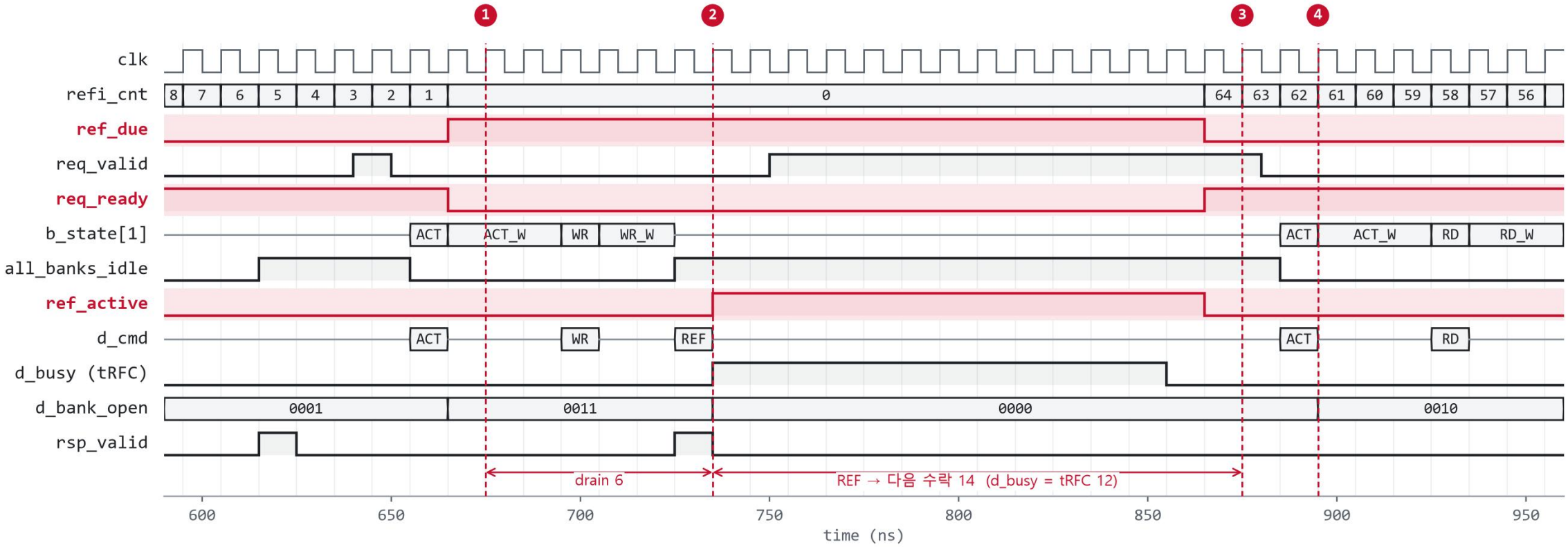
2 1325 ns b0 · b2가 함께 ACT 대기 → `rr_ptr` = 2라 b2 먼저, b0는 다음 클럭. 132–139클럭 매 클럭 명령

3 1365 ns 첫 WR는 closed인 b2 — PRE부터 해야 하는 b0 · b1(conflict)보다 먼저 끝남

4 1405 ns 응답은 tag 순서 b0 → b1 → b2 → b3 연속 4클럭. 먼저 끝난 b2는 버퍼에서 3클럭 대기 · 총 17클럭

tb_mem_ctrl 시나리오 4, 1240–1450 ns · v0는 같은 트래픽에 35클럭 · TB의 '열린 뱅크 ≥ 2' 검사는 버스트 직전에 이미 0011이라 병렬성의 증거가 아님 (19쪽 C)

리프레시: 6클럭 drain → REF → 14클럭 뒤 수락, 열린 row는 닫힌다



1. 675 ns refi_cnt 0 → `ref_due`, 같은 클럭부터 req_ready 0. bank 1은 WR 진행 중
2. 735 ns bank 1 WR 완료 · 전 뱅크 IDLE · 큐 0 → REF 발행 (drain 6). `d_busy` 12클럭 = tRFC
3. 855 → 875 ns busy 해제 → `ref_active` 0 · refi_cnt 64 → 기다리던 RD b1 r33 수락
4. 895 ns REF가 bank_open을 0000으로 → 방금 쓴 row 33도 ACT부터: hit 6이 아니라 closed 10클럭

tb_mem_ctrl 시나리오 3 구간, 590–960 ns · REF 창(수락 차단) 20 · 14 · 14 · 9(잘림)클럭, 전체 312클럭 중 57클럭 = 18.3% (문서 추정 ≈19%)

성능: 버스트 처리량 2.06×, 대신 단독 요청의 지연은 +2–3클럭

같은 TB 트래픽을 v0 · v1 · opt(대기 preload −1, 스크래치 패치)로 돌려 클래스별 수락 → 응답 클럭을 비교

클래스 (횟수)	문서	v0	v1	opt
hit RD (4)	4–5	4	6	6
closed RD (5)	7–9	7	10	9
closed WR (5)	6–8	6	9–12	8–10
conflict RD (1)	10–13	11	14	12
conflict WR (3)	9–12	10	13–14	11–12
4뱅크 버스트 span	–	35	17	15

클럭 · 문서 = scheduler-throughput.md §2 추정 (v0 기준) · write hit는 TB에 없음

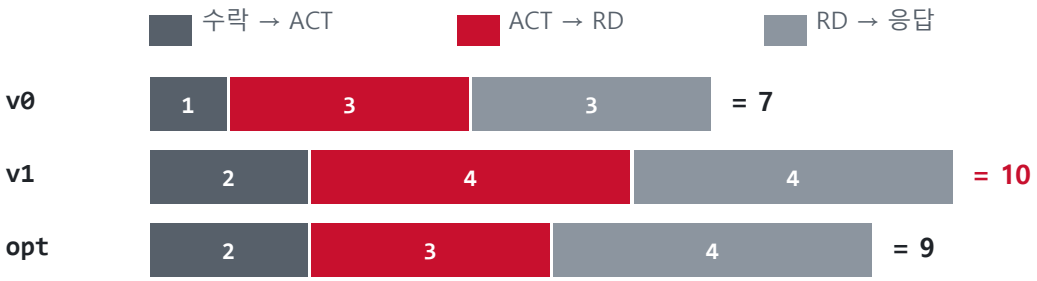
mem_ctrl.sv · 대기 타이머 preload — scratch patch · repo 미반영

```
-287  b_timer[win_bank]   <= 8'(tRP > 0 ? tRP - 1 : 0);
+287  b_timer[win_bank]   <= 8'(tRP > 1 ? tRP - 2 : 0);

-295  b_timer[win_bank]   <= 8'(tRCD > 0 ? tRCD - 1 : 0);
+295  b_timer[win_bank]   <= 8'(tRCD > 1 ? tRCD - 2 : 0);
```

- opt: tRCD · tRP 측정 최소 3 · 3 = 스펙, 위반 0, 저장소 TB PASS
- 면적 9,774 μm² · 2.644 ns — 비용 거의 없음. tRCD/tRP = 1이면 다시 +1

closed RD 하나를 쪼개면 (txn 14 · RD b0 row 10)



+1 큐 단계

수락 클럭에 큐에 쓰고, 다음 클럭에 dispatch → 엔진이 ACT 요청. v0는 IDLE에서 바로 래치

+1 대기 → 발행

timer 0인 클럭엔 상태만 바뀌고 발행은 다음 클럭 중재기에서. t−1 preload는 v0 기준 (opt가 제거)

+1 응답 버퍼

data_valid → `rsp_done` 레지스터 → 다음 클럭 rsp_valid. in-order 재정렬의 비용

처리량

4뱅크 버스트 35 → 17클럭 (2.06×). 132–139클럭 동안 매 클럭 명령 1개 — 이 구간은 명령 버스가 병목. 엔진 2개 이상 동시 활성 11클럭

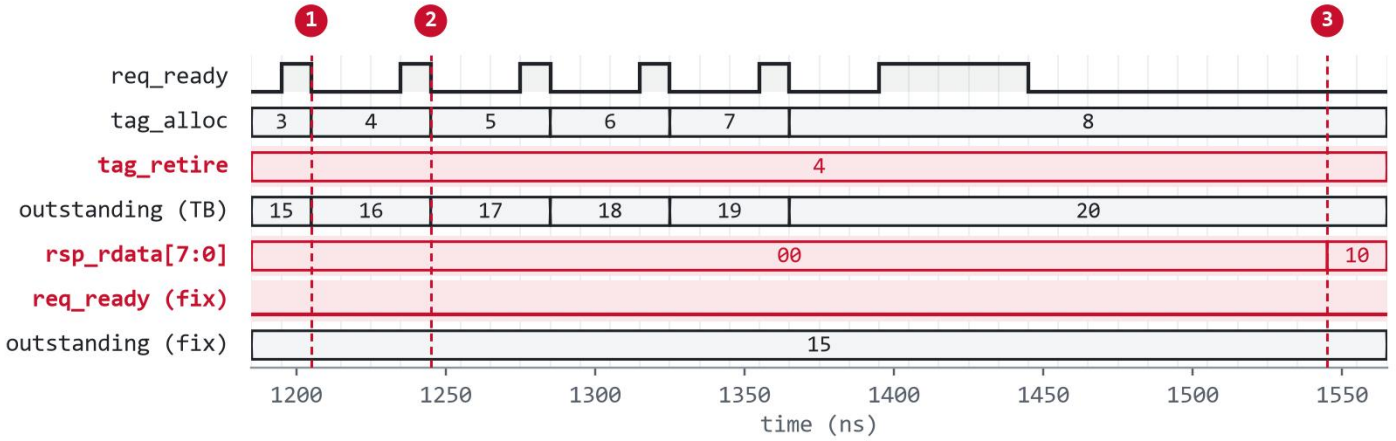
리프레시 · 읽기 파이프

REF 창이 시간의 18.3% (문서 ≈19%) · RD는 `rd_inflight` 하나라 최대 4클럭에 1번 (백프레셔 TB에서 RD 20개 간격 실측) — 연속 읽기 대역의 상한

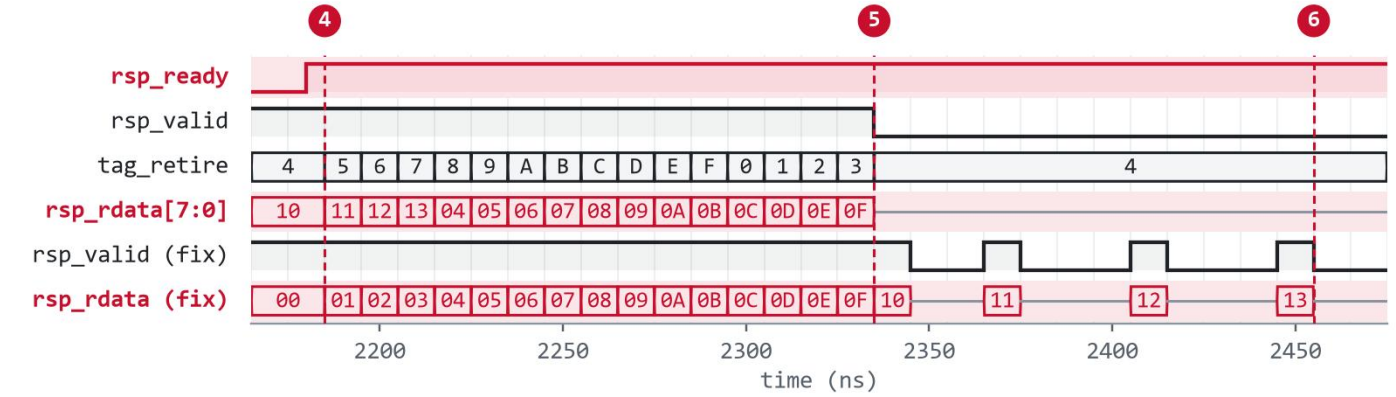
버그: rsp_ready를 오래 막으면 4비트 tag가 한 바퀴 돌아 응답을 덮어쓴다

docs/coverage.md의 gap 'Backpressure'를 실험 TB로 재현 — 읽기 20개를 연달아 보내고 rsp_ready를 150클럭 동안 0

① 수락: tag 4가 아직 반환 전인데 rd16에게 다시 할당



② rsp_ready 해제: 앞 4개는 남의 데이터, 뒤 4개는 사라짐



원인

`req_ready`는 큐 빈 슬롯만 본다 (L113). 큐는 dispatch로 계속 비므로 응답이 막혀도 수락이 이어지고, `tag_alloc`이 16을 돌아 아직 반환 안 된 `tag_retire`(4)를 다시 준다. RD 완료는 `rsp_data[b_tag]`를 빈칸 확인 없이 쓴다 (L337).

1. 1205 ns outstanding 15 → fix는 req_ready 0으로 멈춤, head는 계속 수락
2. 1245 ns rd16이 tag 4를 받음 = tag_retire (rd0 미반환)
3. 1545 ns rd16 완료가 `rsp_data[4]`를 00 → 10으로 덮어씀
4. 2185 ns 해제 후 첫 응답: rd0 자리에 rd16의 데이터 10
5. 2335 ns 16개 반환 뒤 rsp_valid 0 — rd16–19 응답 없음
6. 2455 ns fix: 00 … 13, 20개가 순서대로

fix · mem_ctrl.sv L113 — 스크래치 패치, 저장소 미반영

```
- req_ready = free_found && !ref_due && !ref_active;
+ req_ready = free_found && !ref_due && !ref_active &&
+              (TAG_BITS'(tag_alloc + 1'b1) != tag_retire);
```

head: 응답 16/20, 불일치 4 + 누락 4 → FAIL · fix: 20/20 PASS, 저장소 TB도 PASS · 면적 9,692 μm² · reg2reg 2.880 ns

이슈 기록: 커밋 히스토리에서 확인한 3건 + AI 보조 재검증 제안 5건

왼쪽은 git log · diff · 재빌드로 확인되는 변경, 오른쪽은 재실행 · 실험 · 코드 리딩에서 새로 본 항목 (제안, 저장소 수정 없음)

커밋 히스토리 → 원인 → 조치

이슈	원인 · 근거	조치
64e73e0 모델 컴파일 실패	`row_match` · `legal_access` 미선언 (L107–108) — 이번에 Verilator 에러 2건으로 재현	`6049e5a`에서 식을 인라인. v0 A/B는 HEAD 모델 + v0 mem_ctrl로 측정
64e73e0 access_err 도입	PRE 누락 · 잘못된 row 접근을 모델이 플래그하고 TB가 횟수를 셈	현재 TB에서 0회. 단 발화 경로는 미검증 (E)
6049e5a 읽기 파이프 1개	모델의 CAS 데이터 파이프가 하나 (coverage.md)	`rd_inflight`로 RD 직렬화 → RD 4클럭에 1번 (17쪽)

문서 일부(spec §4/§5, scheduler-throughput.md 머리말)는 아직 v0 구조 기준 — 3쪽 참고
이 덱의 수치는 모두 v1 RTL을 다시 돌려 잰 값

AI 보조 재검증 발견 (제안 · 미반영)

A 4비트 tag 랩어라운드 (18쪽)

응답 버퍼 16칸인데 수락 한도가 없음 → 덮어쓰기 · 누락. 제안: `tag_alloc + 1 ≠ tag_retire`로 수락 제한 (실험 PASS)

B 대기 → 발행 handoff +1 (17쪽)

tRCD · tRP 스펙 3, 실측 최소 4. 위반은 아니지만 클래스마다 1–2클럭 손해. 제안: preload t–2 또는 종료 클럭 발행

C 병렬성 검사가 증거가 아님

TB의 '열린 뱅크 ≥ 2'는 버스트 직전 0011로 이미 만족, v0도 PASS. 제안: 동시 활성 엔진 수나 버스트 클럭 수로 판정

D 큐 깊이 · write hit 미검증

blocking 태스크라 큐 점유 최대 1, L259 · L271 미실행. 제안: 응답을 기다리지 않는 연속 요청 + 같은 row WR

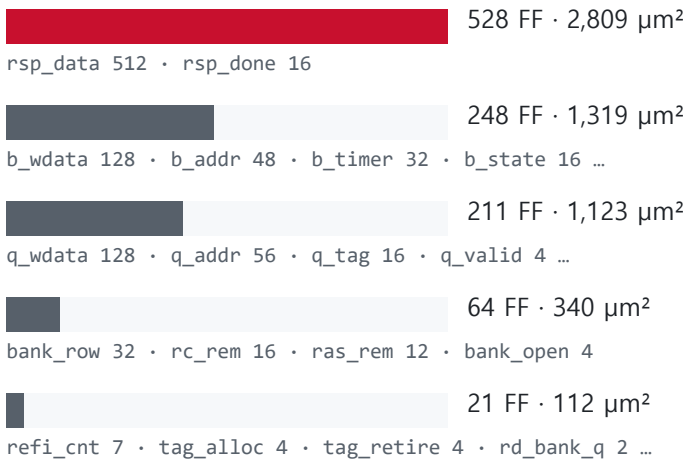
E 모델 체커가 한 번도 발화하지 않음

L110 · L133 분기 미실행 → access_err 경로를 시험한 적 없음. 제안: PRE 누락 · 잘못된 row 음성 테스트, 타이밍 assertion

PPA: 면적 58%가 플롭, 그 절반이 응답 버퍼 — 임계 경로는 tag 나이 비교

sv2v → Yosys 0.52 → Nangate45 typical (OpenSTA 예제 라이브러리) → OpenSTA 3.1 · mem_ctrl만 · 배선 · 클럭 트리 없음

플롭 1,072개의 용도 (전부 DFFR/DFFS · 5.32 μm²)



순차 5,703 μm² = 전체 9,819의 58% · 응답 버퍼만 29% · 데이터를 담는 플롭 768개 = FF의 72%

면적 레버 (미측정 · 셀 면적으로 추정)

- 데이터 플롭 768개는 리셋 불필요 → DFF_X1(4.522)이면 약 −613 μm²
- `b_timer` 8b × 4 → 2b면 충분 (preload 최대 2) → −24 FF
- `b_wdata` 128 FF는 `q_wdata`의 복사 — 슬롯 인덱스만 넘기면 제거 가능
- tag 수 = 응답 버퍼 깊이 — fix의 수락 한도와 함께 줄이면 비례 감소

	v0	v1 head	fix	opt
면적 μm²	1,208	9,819	9,692	9,774
셀	559	4,414	4,433	4,384
FF	131	1,072	1,072	1,072
reg2reg ns	0.809	2.642	2.880	2.644
≈ fmax MHz	1,182	373	343	374
전력 mW *	1.63	18.26	18.31	17.94

임계 경로 (reg2reg, v1 head)

- `tag_retire[1]` → `q_tag − tag_retire` 나이 비교 → 슬롯 선택 → page 판정 → `b_state[2][2]` · 29단 2.642 ns. 팬아웃 46 NOR4 한 단이 0.866 ns (33%)
- reg2out 1.615 ns (`rr_ptr[1]` → `dram_row[1]`, 명령 출력 레지스터 없음) · in2reg 1.677 ns (`dram_busy`)

읽는 법

Nangate45는 공개 교육용 라이브러리 · 배선 부하 · 클럭 트리 없음 · *전력은 500 MHz, 입력 토글 0.1 가정 (벡터 없음), 순차 47.6% / 조합 52.4%. 절대값이 아니라 v0 · v1 · 패치끼리의 상대 비교용이다. 저장소 문서의 PPA는 지연 · 처리량만 다룬다.

AI 활용: 초안 · 분석은 AI에 맡기고, 작업 지시와 결과 확인은 직접

실무에서 LLM을 쓴 방식과, 이 포트폴리오를 만든 파이프라인을 구분해 적는다

실제 사용 — 실무와 이 포트폴리오

실무 (SoC 디지털 설계) · Claude Opus · GPT

- I2C Master IP RTL: FSM · 레지스터 뼈대를 LLM 초안으로 빠르게 구현
- Chip Top UVM 환경: Driver · Monitor · Sequence 코드 생성 자동화
- 설계 초안 작성 · 검증 셋업 시간 2배 이상 단축

이 포트폴리오 · Cursor AI 에이전트 + 스크립트

- AI 에이전트: 재실행 · VCD 파형 분석 · 합성 리포트 · 덱 생성
- 본인: 작업 지시와 결과(숫자 · 파형 · 리포트) 확인

원칙 (이 덱에 적용한 것)

- 숫자는 손으로 옮기지 않는다: 시뮬 · 합성 출력 파일 → 스크립트 → 슬라이드
- 측정할 수 없는 것은 '미측정 · 추정 · 확인 필요'로 표시
- AI가 제안한 수정은 스크래치에서만 시험하고, PASS/FAIL로 판정

이 포트폴리오를 만든 파이프라인 (Cursor AI 에이전트 + 스크립트)

1	저장소 고정	commit `6049e5a` = origin/main · git status clean · 소스 수정 없음
2	Verilator 재실행	Makefile과 같은 플래그 · `--trace` 래퍼 · `--coverage` → log · VCD · coverage.dat
3	VCD 분석	명령 간격 7규칙 · 요청별 page 클래스 · 지연 분해 · 엔진 동시성 (analyze.py)
4	A/B · 실험	v0 (64e73e0) · opt · fix 패치와 백프레셔 TB — 스크래치 폴더에서만
5	합성 · STA	sv2v → Yosys → Nangate45 → OpenSTA · 플롭을 RTL 레지스터로 역추적해 집계
6	덱 생성 · 리뷰	pygments 코드 캡처 · matplotlib 파형 · python-pptx 도형 · 전 슬라이드 렌더 검토

19쪽 발견 A–E와 패치는 분석 중 AI 에이전트가 제안 — 스크래치에서만 시험, 저장소 수정 없음

12 마무리

JD 연결: DRAM Memory Controller를 설계하고, 클럭 단위로 분석한다

SK hynix HBM Digital Design (RTL Design) — 담당 업무 · 우대 사항과 이 프로젝트의 근거

JD 항목	이 프로젝트의 근거	쪽
DRAM Memory Controller 설계	깊이 4 큐 · oldest-tag dispatch · 뱅크 엔진 ×4 · PRE > ACT > WR > RD 중재 · drain-then-REF	5–11
DRAM Memory Controller 분석	클래스별 지연 분해 (+1 원인 3개) · 버스트 2.06× · 리프레시 18.3%	14–17
Spec · PPA → Architecture	주소 맵 · 타이밍 8개 고정, v0 → v1을 지연 · 처리량 · 면적으로 비교	3–4, 20
RTL 설계 · Function 검증	저장소 TB PASS + VCD 타이밍 체커 위반 0 · 커버리지 구멍 → 실험 TB	12–13
이슈 분석 · 디버그	tag 랩어라운드 재현 → 수정안(스크래치) 20/20 PASS · AI 보조 재검증 제안 5건	18–19
PD Guide · PPA	Nangate45 면적 · 타이밍 · 전력 참고치, 임계 경로와 면적 레버	20
DRAM DFT / DFD · Wafer Test	모델 관측 포트 (access_err · bank_open)까지 — 테스트 모드 · BIST는 범위 밖	5, 11
High Speed PHY · Full-custom	범위 밖 — High Speed Link Lite 포트폴리오에서 다룸	–
LLM / AI 기반 EDA 자동화	실무: LLM으로 RTL 뼈대 · UVM 코드 생성, 초안 · 셋업 시간 2배 이상 단축 이 덱: AI 에이전트로 재실행 · VCD 분석 · 합성 리포트 · 덱 생성	21

다음 단계

- fix 반영 + 백프레셔 · REF-under-load 회귀 추가
- 대기 종료 클럭 발행으로 handoff +1 제거
- non-blocking 호스트 + 랜덤 트래픽으로 큐 깊이 · write hit
- 모델에 타이밍 assertion · 음성 테스트
- 응답 버퍼 · `b_timer` 축소 후 면적 재측정

한 줄 요약

타이밍을 지키는 컨트롤러를 만들고, 그 컨트롤러가 얼마나 빠른지 · 어디서 깨지는지를 파형과 숫자로 설명한다.

용현 권

kyongh0331@naver.com

github.com/yhkwon-git/mini-dram-controller